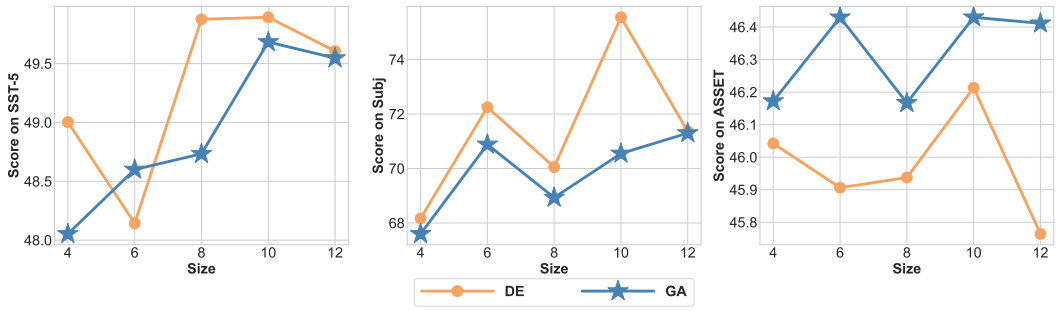


Figure 6: Effect of population size on SST-5 (left), Subj (middle), and ASSET (right). All the results are averaged over 3 random seeds.

Text Classification The population of prompts is initialized with widely used instructions in the previous works (Mishra et al., 2022b; Zhang et al., 2022). We paraphrase and rewrite them to initialize the population. The size of the development set is 200. We report the results on the full test set (the same as the previous related works (Deng et al., 2022; Zhang et al., 2023a)), as shown in Table 11.

Task LM	|Population|	|Steps|	|Dev|	|Shots|
Text classification				
Alpaca-7b	10	10	200	1
Text Generation				
Alpaca-7b	10	10	100	0
GPT-3.5	10	10	100	0
Big-Bench Hard				
GPT-3.5	10	10	50	3

Table 11: Settings for experiments. |Shots| refers to the number of examples in the demonstration. For the text classification task, we set the value as 1, which means we prepend with 1 sample of each category, to constrain the output in the label space.

Text Generation For the initial population, we collect instructions for summarization and simplification from Li et al. (2023); Sanh et al. (2021); Zhang et al. (2023c) and augment them to the expected size (10 in our setting), either written manually or generated by GPT-3.5.

C Additional Results

C.1 Parameters in Evolutionary Algorithms

Effect of Population Size Intuitively, a trade-off exists between the performance and the overhead caused by the population size. We explore the performance of EvoPrompt (DE) and EvoPrompt (GA) respectively at varying population sizes from 4 to 12. The results are plotted in Figure 6.

For classification datasets, as the size increases, curves for DE and GA show an ascending trend. Furthermore, the increase in DE attributed to population diversity was greater than that in GA since DE focuses on different parts. Differences among prompts within populations bring about substantial mutations, leading DE to explore potential prompts since keeping common parts balances exploration and exploitation effectively.

For the relatively simple generation task (i.e., ASSET), a population size of 6 demonstrates a comparable performance to a population size of 10, though with a 2.5-fold increase in overhead. This suggests that for relatively simple tasks large populations are unnecessary, while for complex tasks (i.e., Subj), a larger population with diversity brings improvement.

Effect of Number of Iterations To further explore the process of convergence, for SST-5, Subj and ASSET, we plot the best and average scores on the development set for EvoPrompt for DE and GA over the whole population after each iterative step (Figure 7). Curves of best and average scores gradually converge with an increasing trend as evolution proceeds, indicating that the population's quality as a whole is steadily increasing as the evolution process.

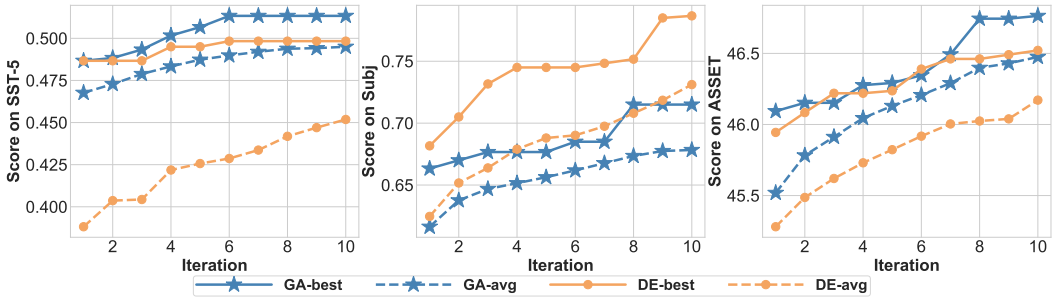


Figure 7: The best and average scores of each iteration on SST-5 (left), Subj (middle), and ASSET (right) development set on Alpaca-7b. All the results are averaged over 3 random seeds.

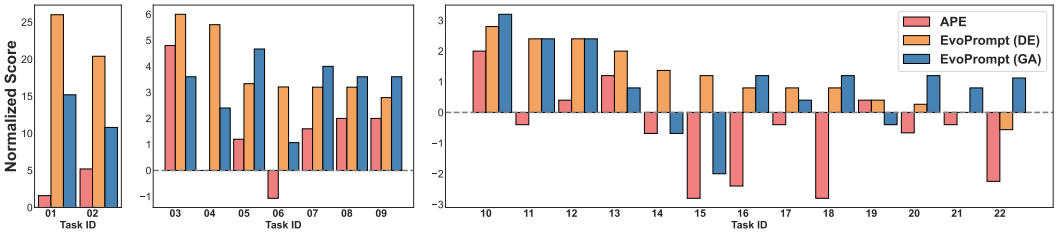


Figure 8: Normalized scores on BBH tasks for APE, EvoPrompt (GA) and EvoPrompt (DE).

C.2 Comparison on BBH Tasks

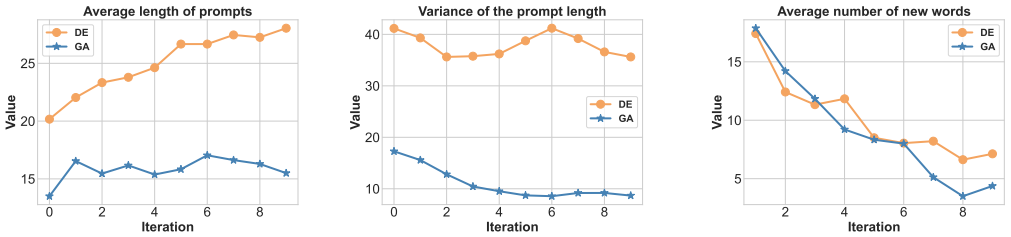
APE (Zhou et al., 2022) optimizes the Chain-of-Thought (CoT) prompt for reasoning tasks on InstructGPT. Considering that both InstructGPT and GPT-3.5 belong to the GPT family and we may observe similar trends, we evaluate the CoT prompt proposed by APE, "Let's work this out in a step by step way to be sure we have the right answer.", on reasoning tasks and plot the 3-shot performance in Figure 8. For simplicity, we use the same initial population for all the 22 BBH tasks without priori knowledge of each task. In future works, by incorporating task-specific prompts, either manually designed or generated by LLMs, we may further enhance the performance.

Method	**Avg.**
baseline	71.49
APE	71.85
EvoPrompt (GA)	74.18
EvoPrompt (DE)	**75.03**

Table 12: Average accuracy over 23 BBH tasks for different methods.

	SST-5			**Subj**		
	APE	EvoPrompt (GA)	EvoPrompt (DE)	APE	EvoPrompt (GA)	EvoPrompt (DE)
Same iteration						
# iterations	9	9	9	15	15	15
# tokens	5.39 M	5.40 M	5.52 M	5.66 M	5.73 M	5.93 M
score	45.79	50.23	49.23	67.20	70.10	79.35
Until convergence						
# iterations	9	7	11	15	15	17
# tokens	5.39 M	4.20 M	6.75 M	5.66 M	5.73 M	6.72 M
score	45.79	50.23	51.13	67.20	70.10	79.35

Table 13: Number of iterations, tokens within the API requests (including prompt optimization and evaluation) and the corresponding score for our methods and APE. We choose the iteration that APE converges as the *Same iteration* for comparison. *Until convergence* means that the improvement of the average score is less than 0.3% for continuous two iterations.



(a) Average length over the population after each step.

(b) Variance of prompt length over the population of each step.

(c) Number of new words generated after each step.

Figure 9: Statistics about the prompt length, including average values over the whole population (a), variance over the prompt length (b), and number of new words evolved after each step (c). Note that all the values are averaged over 8 datasets, including 7 understanding datasets and one simplification dataset, and 3 random seeds.

C.3 Cost Analysis

Overhead mainly comes from prompt evaluation and generation. For evaluation, our overhead is $N * |D| * T$, where N is the size of the population, $|D|$ is the size of the development set, and T is the number of iterations. These parameters differ from the task and can be found in Appendix B.3. For the cost from prompt generation, the cost mainly depends on the number of API results, $T * N$. So the total number of API requests is $N * T * (1 + |D|)$, the same as APE. Moreover, given that the API of LLMs is typically billed based on the number of tokens used, we also estimate the total number of tokens used in the API requests during the prompt optimization process, as shown in Table 13. All the scores reported are over the test set on one random seed. We analyze the overhead mainly from two aspects: 1) the performance of our methods compared with APE under the *same number of iterations*; 2) the performance *until convergence* measured by the average score on the dev set.

We can observe that with the same number of iterations, both GA and DE outperform APE significantly while introducing only a slight overhead in terms of the number of tokens. The convergence rates of APE and GA are similar while DE is slightly slower, but it delivers better performance. This implies the relatively high ceiling of EvoPrompt.

C.4 Analysis of Prompt

Diversity Analysis We further investigate the diversity of prompts generated by GA and DE after each iterative step respectively. We mainly plot the average prompt length, variance and number of new words mutated after each step, as shown in Figure 9. It can be observed that EvoPrompt (DE) generates longer prompts with higher variances than EvoPrompt (GA), which implies that DE prefers exploration for diversity. In the latter iterations, DE mutates more new words than GA, and thus shows better potential to escape from the local optimum.

Optimal Prompts We release the optimal prompts generated by EvoPrompt for understanding (Table 14), text simplification (Table 16), summarization (Table 15) and BBH tasks (Table 17, 18) .

D Future Works

There are several promising directions for future investigation:

- Based on our framework, more applications can be explored, including game levels generation, text-to-images generation, non-trivial NP-hard problems (e.g. traveling salesman problem), etc.
- There exist many variants of DE and we give priority to the most canonical and classical ones for current exploration. In future work, it will be interesting to consider more advanced DE-variants (Das et al., 2016; Das & Suganthan, 2010). For example, some recent DE-variants have been investigating adaptive control parameters. The main challenge in applying these variants to